Prisma Press Backend Requirements

1. Purpose

Prisma Press is a modular blog backend built with Express, TypeScript, Prisma, and PostgreSQL. It provides the API needed for authentication, user profiles, blog posts, comments, and admin reporting.

This document is the source of truth for the backend. It defines what the system must do, how the API behaves, and what limits already exist in the codebase.

2. Scope

The backend must support:

- User registration, login, and token refresh.
- Logged-in user profile retrieval and profile updates.
- Post creation, listing, search, update, delete, and stats.
- Comment creation, retrieval, ownership checks, and moderation.
- Role-based access control for normal users and admins.

3. Technology Stack

The backend must run on the following stack:

Area	**Requirement**	**Notes**
Runtime	Node.js	Development uses `tsx`; production uses compiled TypeScript output.
Language	TypeScript	Strict compiler settings are enabled.
Web framework	Express 5	All routes are mounted in one Express application.
ORM	Prisma 7.x	Uses the generated client in `generated/prisma`.
Database	PostgreSQL	Connected through `@prisma/adapter-pg`.
Authentication	JWT + bcrypt	Access and refresh tokens are issued after password verification.
Middleware	cors, cookie-parser, body parsers	Cookies are used for token storage.

4. Package Manager Support

The project supports both pnpm and npm.

- Install dependencies with `pnpm install` or `npm install`.
- Start development with `pnpm dev` or `npm run dev`.
- Build the project with `pnpm build` or `npm run build`.
- Run the compiled server with `pnpm start` or `npm start`.

5. Environment and Startup

The application reads configuration from `.env`.

Required environment variables:

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string.
`PORT`	HTTP server port. Defaults to `3000` if missing.
`APP_URL`	Allowed browser origin for CORS.
`BCRYPT_SALT_ROUNDS`	Password hashing cost.
`JWT_ACCESS_SECRET`	Access-token signing secret.
`JWT_REFRESH_SECRET`	Refresh-token signing secret.
`JWT_ACCESS_EXPIRES_IN`	Access-token lifetime.
`JWT_REFRESH_EXPIRES_IN`	Refresh-token lifetime.

Startup requirements:

1. Load environment variables.
2. Connect to Prisma before starting the server.
3. Start listening only after the database connection succeeds.
4. Disconnect Prisma and exit with an error if startup fails.

The server must also expose a root route at `/` that returns `Hello, World!`.

6. API Conventions

The backend accepts JSON and URL-encoded request bodies and reads cookies on every request.

6.1 Protected Requests

Protected routes require a valid access token in the `Authorization` header.

The current middleware reads the raw token value directly, so clients must send the token exactly as expected by the app.

Example:

```
Authorization: <access-token>
Content-Type: application/json
```

6.2 Standard Response Shape

Most successful responses use this shape:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Request completed successfully",
  "data": {}
}
```

Paginated responses may also include `meta`:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Posts retrieved successfully",
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 42
  },
  "data": []
}
```

Error responses should remain readable and should include an error message and failure details.

7. Authentication

Authentication uses access tokens and refresh tokens.

Method	Route	Access	Behavior
`POST`	`/api/auth/login`	Public	Verifies email and password, then returns access and refresh tokens. Also sets both tokens as HTTP-only cookies.
`POST`	`/api/auth/refresh-token`	Public	Reads the refresh token from cookies and returns a new access token.

7.1 Login

Example request:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Example response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "User logged in successfully",
  "data": {
    "accessToken": "eyJhbGciOi...",
    "refreshToken": "eyJhbGciOi..."
  }
}
```

7.2 Refresh Token

Example request:

```
POST /api/auth/refresh-token
Cookie: refreshToken=eyJhbGciOi...
```

Example response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Token refreshed successfully",
  "data": {
    "accessToken": "eyJhbGciOi..."
  }
}
```

7.3 Cookie Rules

- `refreshToken` lifetime: 7 days.
- `accessToken` lifetime: 1 hour.
- Cookies are HTTP-only.
- Current implementation uses `sameSite: 'none'` and `secure: false`.

8. User Management

Method	Route	Access	Behavior
`POST`	`/api/users/register`	Public	Creates a new user and an associated profile in a single transaction.
`GET`	`/api/users/me`	Authenticated `USER` or `ADMIN`	Returns the logged-in user profile without the password.
`PUT`	`/api/users/my-profile`	Authenticated `USER` or `ADMIN`	Updates the current user profile.

8.1 Register User

Example request:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123",
  "profilePhoto": "https://example.com/photo.jpg"
}
```

Example response:

```
{
  "success": true,
  "statusCode": 201,
  "message": "User registered successfully",
  "data": {
    "id": "user-id",
    "name": "John Doe",
    "email": "john@example.com",
    "activeStatus": "ACTIVE",
    "role": "USER",
    "profile": {
      "profilePhoto": "https://example.com/photo.jpg",
      "bio": null
    }
  }
}
```

8.2 Profile Update

Example request:

```
{
  "name": "John Updated",
  "profilePhoto": "https://example.com/new-photo.jpg",
  "bio": "Software Engineer and Writer"
}
```

Example response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "User profile updated successfully",
  "data": {
    "id": "user-id",
    "name": "John Updated",
    "email": "john@example.com",
    "profile": {
      "profilePhoto": "https://example.com/new-photo.jpg",
      "bio": "Software Engineer and Writer"
    }
  }
}
```

8.3 User Rules

- Email addresses must be unique.

- Passwords must be hashed with bcrypt.
- New users default to `ACTIVE` status and `USER` role.
- A profile record must be created with every user.

9. Posts

Method	Route	Access	Behavior
`GET`	`/api/posts`	Public	Returns a filtered, paginated list of posts.
`GET`	`/api/posts/stats`	`ADMIN` only	Returns aggregate content and user statistics.
`GET`	`/api/posts/my-posts`	Authenticated `USER` or `ADMIN`	Returns posts created by the logged-in user.
`GET`	`/api/posts/:postId`	Public	Returns a single post and increments its view count.
`POST`	`/api/posts`	Authenticated `USER` or `ADMIN`	Creates a new post for the logged-in author.
`PATCH`	`/api/posts/:postId`	Authenticated `USER` or `ADMIN`	Updates a post with ownership rules.
`DELETE`	`/api/posts/:postId`	Authenticated `USER` or `ADMIN`	Deletes a post with ownership rules.

9.1 Create Post

Example request:

```
{
  "title": "My First Post",
  "content": "Content of the post goes here.",
  "thumbnail": "https://example.com/thumbnail.jpg",
  "isFeatured": false,
  "status": "PUBLISHED",
  "tags": ["typescript", "prisma", "express"]
}
```

Example response:

```
{
  "id": "post-id",
  "title": "My First Post",
  "content": "Content of the post goes here.",
  "thumbnail": "https://example.com/thumbnail.jpg",
  "isFeatured": false,
  "status": "PUBLISHED",
  "tags": ["typescript", "prisma", "express"],
  "views": 0,
  "authorId": "user-id"
}
```

9.2 List and Search Posts

The post list endpoint must support these query parameters:

- `search`
- `tags`
- `isFeatured`
- `status`
- `authorId`
- `page`
- `limit`
- `sortBy`
- `sortOrder`

Defaults:

- `page = 1`
- `limit = 10`
- `sortBy = createdAt`
- `sortOrder = desc`

Example request:

```
GET /api/posts?
search=prisma&tags=typescript,backend&page=1&limit=10&sortBy=createdAt&sortOrder
=desc
```

Example response:

```
{
  "data": [
    {
      "id": "post-id-1",
      "title": "Prisma with Express",
      "views": 12,
      "_count": {
        "comments": 3
      }
    }
  ],
  "pagination": {
    "total": 1,
    "page": 1,
    "limit": 10,
    "totalPages": 1
  }
}
```

9.3 Single Post

Reading a post must increment its view count and return approved comments only.

Example response:

```
{
  "id": "post-id-1",
  "title": "Prisma with Express",
  "content": "Full post content...",
  "views": 13,
  "comments": [
    {
      "id": "comment-id-1",
      "content": "Great article!",
      "status": "APPROVED"
    }
  ],
  "_count": {
    "comments": 1
  }
}
```

9.4 Ownership Rules

- Admins may update or delete any post.
- Regular users may update or delete only their own posts.
- Regular users may not change `isFeatured`.

9.5 Stats

The stats endpoint must return totals for:

- Total posts.
- Published posts.
- Draft posts.
- Archived posts.
- Total comments.
- Approved comments.
- Total users.
- Admin users.
- Regular users.
- Total views.

10. Comments

Method	Route	Access	Behavior
`GET`	`/api/comments/author/:authorId`	Public	Lists comments written by a specific author.
`GET`	`/api/`	Public	Returns a single

Method	Route	Access	Behavior
	`comments/:commentId`		comment with selected post information.
`POST`	`/api/comments`	Authenticated `USER` or `ADMIN`	Creates a comment for a post.
`PATCH`	`/api/comments/:commentId`	Authenticated `USER` or `ADMIN`	Updates the current user's own comment.
`DELETE`	`/api/comments/:commentId`	Authenticated `USER` or `ADMIN`	Deletes the current user's own comment.
`PATCH`	`/api/comments/:commentId/moderate`	`ADMIN` only	Changes comment moderation status.

10.1 Create Comment

Example request:

```
{
  "content": "This is a comment",
  "postId": "post-id-1"
}
```

Example response:

```
{
  "id": "comment-id-1",
  "content": "This is a comment",
  "authorId": "user-id",
  "postId": "post-id-1",
  "status": "APPROVED"
}
```

10.2 Comment Rules

- Anyone can read comments.
- Logged-in users can create comments.
- Only the comment owner can edit or delete.
- Only admins can moderate.
- A comment can only be created for an existing post.
- New comments default to `APPROVED`.
- Moderation changes can set the status to `APPROVED` or `REJECT`.

10.3 Moderation Example

```
{
  "status": "REJECT"
```

```
}
```

11. Data Model Summary

11.1 User

- UUID primary key.
- Fields: `name`, `email`, `password`, `activeStatus`, `role`, `createdAt`, `updatedAt`.
- Email must be unique.
- Defaults: `ACTIVE` status and `USER` role.
- Relations: one `Profile`.

11.2 Profile

- UUID primary key.
- `userId` is unique.
- Cascade delete when the user is deleted.
- Optional `profilePhoto` and `bio` fields.

11.3 Post

- UUID primary key.
- Fields: `title`, `content`, `thumbnail`, `isFeatured`, `status`, `tags`, `views`, `authorId`.
- Default status is `PUBLISHED`.
- `authorId` is indexed.

11.4 Comment

- UUID primary key.
- Fields: `content`, `authorId`, `postId`, `status`.
- Cascade delete when the post is deleted.
- `postId` and `authorId` are indexed.

12. Error Handling and Security

12.1 Error Handling

The backend must keep the existing error behavior:

- 404 responses for unknown routes.
- Global error handling for unhandled errors.

- Prisma errors translated into readable HTTP responses.

12.2 Security

- Passwords must be hashed before storage.
- Protected routes must require a valid JWT.
- Role checks must be enforced where needed.
- Token cookies must be HTTP-only.

13. Current Constraints

The current codebase has these constraints:

- `APP_URL` is required for CORS but is not documented in the sample environment file.
- npm support is available through the existing scripts.
- The auth middleware reads the raw `Authorization` header value.
- The schema defines an `AUTHOR` role, but the current routes mainly use `USER` and `ADMIN`.
- `Post.authorId` and `Comment.authorId` are stored as strings rather than explicit user relations.
- No validation library, rate limiting, or automated tests are implemented.
- Logging is mostly done with `console`.
- Some controllers return helper-based responses while others return custom JSON.

14. Endpoint Summary

Area	Method	Route	Access
Auth	`POST`	`/api/auth/login`	Public
Auth	`POST`	`/api/auth/refresh-token`	Public
Users	`POST`	`/api/users/register`	Public
Users	`GET`	`/api/users/me`	`USER` or `ADMIN`
Users	`PUT`	`/api/users/my-profile`	`USER` or `ADMIN`
Posts	`GET`	`/api/posts`	Public
Posts	`GET`	`/api/posts/stats`	`ADMIN` only
Posts	`GET`	`/api/posts/my-posts`	`USER` or `ADMIN`
Posts	`GET`	`/api/posts/:postId`	Public

Area	Method	Route	Access
Posts	POST	/api/posts	USER or ADMIN
Posts	PATCH	/api/posts/:postId	USER or ADMIN
Posts	DELETE	/api/posts/:postId	USER or ADMIN
Comments	GET	/api/comments/author/:authorId	Public
Comments	GET	/api/comments/:commentId	Public
Comments	POST	/api/comments	USER or ADMIN
Comments	PATCH	/api/comments/:commentId	USER or ADMIN
Comments	DELETE	/api/comments/:commentId	USER or ADMIN
Comments	PATCH	/api/comments/:commentId/moderate	ADMIN only

15. Delivery Standard

Future work should preserve the existing module structure, keep Prisma as the source of truth for persistence, and maintain the current API contract unless a versioned change is introduced.

The document should stay plain, direct, and beginner-friendly.